materia: Frontend clase: "4" tipo: taller fecha: 2026-11-12 entrega: 2026-11-13 tags: [clase, frontend, taller]

Taller 4 · Terminar el Layout antes de la clase de mañana

Programación IV · Frontend · UNIAJC · 12 de noviembre de 2026 En parejas, trabajo para la casa. **Entrega: viernes 13-nov-2026, antes de empezar la clase.**

Volver a [[Clase 04 - Layout responsive y navegacion]] · Hoja de la clase: [[Taller Clase 04 parte 2 - 2026-11-12]]

> **¿Por qué para mañana?** En la clase de mañana (Clase 5) las tarjetas del catálogo se ponen dentro de este Layout. Si mañana su proyecto no tiene el pie, el botón y las rutas, se quedan atrás desde el primer minuto.

Punto de partida

En clase llegamos hasta `Layout.astro` y `Header.astro`. Antes de seguir, comprueben:

- [ ] `npm run dev` abre la portada **sin errores** en la terminal ni en la pestaña *Problems* de VS Code.
- [ ] La portada ya toma los colores y las fuentes de su `variables.css`.
- [ ] El Header muestra sus enlaces y marca la página activa.

Si algo de esto falla, arréglenlo primero. Revisen la tabla de errores frecuentes al final.

Regla del taller: el código de abajo es el de SaborU, el proyecto de ejemplo. **No se copia, se traduce.** Los textos, la marca, la API del crédito y las rutas salen de **su** briefing y **su** mockup. Las clases CSS y las variables (`--esp-4`, `--color-marca`…) sí pueden quedar iguales si su `variables.css` las tiene.

Paso 1 · `Footer.astro`: Grid de una a tres columnas

Creen `src/components/layout/Footer.astro`:

```
---
// Footer.astro
// Creditos y enlaces secundarios. El credito a la API no es opcional.
const base = import.meta.env.BASE_URL;
const anio = new Date().getFullYear();
---

<footer class="pie">
  <div class="contenedor pie__interior">
    <div class="pie__bloque">
      <p class="pie__marca">SaborU</p>
      <p class="pie__texto">
        Catalogo de recetas rapidas para estudiantes que cocinan solos y con
        poco tiempo.
      </p>
    </div>

    <nav class="pie__bloque" aria-label="Enlaces del pie">
      <p class="pie__titulo">Secciones</p>
      <ul class="pie__lista">
        <li><a href={`${base}catalogo`}>Catalogo</a></li>
        <li><a href={`${base}favoritos`}>Favoritos</a></li>
        <li><a href={`${base}contacto`}>Contacto</a></li>
        <li><a href={`${base}acerca-de`}>Acerca de</a></li>
      </ul>
    </nav>

    <div class="pie__bloque">
      <p class="pie__titulo">Datos</p>
      <p class="pie__texto">
        Las recetas provienen de <a href="https://www.themealdb.com" rel="noopener"
        >TheMealDB</a>, una API publica y gratuita.
      </p>
    </div>
  </div>

  <div class="contenedor pie__legal">
    <p>
      Proyecto academico de Programacion IV — Frontend y Experiencia de
      Usuario · {anio}
    </p>
  </div>
</footer>

<style>
  .pie {
    margin-top: var(--esp-7);
    background: var(--color-fondo);
    border-top: 1px solid var(--color-borde);
  }

  .pie__interior {
    display: grid;
    gap: var(--esp-5);
    padding-top: var(--esp-6);
    padding-bottom: var(--esp-5);
  }
```

```
@media (min-width: 768px) {
  .pie__interior {
    grid-template-columns: 1.6fr 1fr 1.2fr;
    gap: var(--esp-6);
  }
}

.pie__marca {
  font-family: var(--fuente);
  font-size: var(--texto-m);
  font-weight: 600;
  color: var(--color-marca);
  margin: 0 0 var(--esp-2);
}

.pie__titulo {
  font-size: var(--texto-xs);
  letter-spacing: 0.1em;
  text-transform: uppercase;
  font-weight: 700;
  color: var(--color-texto-tenue);
  margin: 0 0 var(--esp-2);
}

.pie__texto {
  margin: 0;
  font-size: var(--texto-s);
  color: var(--color-texto-suave);
  max-width: 42ch;
}

.pie__lista {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: var(--esp-1);
  font-size: var(--texto-s);
}

.pie__lista a {
  color: var(--color-texto-suave);
  text-decoration: none;
}

.pie__lista a:hover {
  color: var(--color-marca);
  text-decoration: underline;
}

.pie__legal {
  border-top: 1px solid var(--color-borde);
  padding-top: var(--esp-3);
  padding-bottom: var(--esp-4);
}
```

```
  .pie__legal p {
    margin: 0;
    font-size: var(--texto-xs);
    color: var(--color-texto-tenue);
  }
</style>
```

Lo que deben cambiar: el nombre de su marca, la descripción, los enlaces (sus rutas) y el crédito a **su** API.

Para entender: un Grid sin columnas definidas es una sola columna. Por eso en el celular los tres bloques se apilan solos, y desde 768 px pasan a tres columnas con proporciones `1.6fr 1fr 1.2fr`.

- [ ] **Listo cuando:** a 1000 px se ven tres columnas y a 375 px una sola.

Antes de seguir: `npm run build`. Si falla con `Unexpected end of input`, tienen un selector mal escrito en un CSS (busquen `:{`). Luego:

```
git add .
git commit -m "feat: layout, header y footer"
```

Paso 2 · `Boton.astro` y `Etiqueta.astro`

Creen `src/components/ui/Boton.astro`:

```
---
// Boton.astro
// Un boton o un enlace con la misma apariencia.
// Props:
//   href       si viene, se dibuja como <a>; si no, como <button>
//   variante   "primario" | "secundario" | "fantasma"
//   tamano     "normal" | "grande"
const {
  href,
  variante = "primario",
  tamano = "normal",
  tipo = "button",
  ...resto
} = Astro.props;

const Etiqueta = href ? "a" : "button";
const clases = ["boton", `boton--${variante}`, `boton--${tamano}`].join(" ");
---

<Etiqueta
  class={clases}
  href={href}
  type={href ? undefined : tipo}
  {...resto}
>
  <slot />
</Etiqueta>

<style>
  .boton {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: var(--esp-2);
    font-family: var(--fuente-texto);
    font-size: var(--texto-base);
    font-weight: 600;
    line-height: 1.2;
    text-decoration: none;
    border-radius: var(--radio-s);
    border: 1px solid transparent;
    cursor: pointer;
    transition: background var(--transicion), border-color var(--transicion),
      transform var(--transicion), box-shadow var(--transicion);
    /* 44 px de alto minimo: es el tamano que un dedo acierta */
    min-height: 44px;
    padding: 0 var(--esp-4);
  }

  .boton--grande {
    min-height: 52px;
    padding: 0 var(--esp-5);
    font-size: var(--texto-m);
  }

  .boton--primario {
    background: var(--color-marca);
```

```
    color: #ffffff;
    box-shadow: var(--sombra-1);
  }

  .boton--primario:hover {
    background: var(--color-marca-oscuro);
    color: #ffffff;
    box-shadow: var(--sombra-2);
  }

  .boton--secundario {
    background: var(--color-fondo);
    color: var(--color-marca);
    border-color: var(--color-marca-borde);
  }

  .boton--secundario:hover {
    background: var(--color-marca-suave);
    color: var(--color-marca-oscuro);
    border-color: var(--color-marca);
  }

  .boton--fantasma {
    background: transparent;
    color: var(--color-texto);
    border-color: var(--color-borde-fuerte);
  }

  .boton--fantasma:hover {
    background: var(--color-superficie);
    color: var(--color-texto);
  }

  .boton:active {
    transform: translateY(1px);
  }
</style>
```

Dos ideas que deben poder explicar mañana:

1. **Si llega `href`, el componente dibuja un `<a>`; si no, un `<button>`.** Un enlace lleva a otra página y un botón hace algo en la misma página. Se ven iguales, pero no son lo mismo.
2. **`min-height: 44px`** es el tamaño que un dedo acierta en el celular. Con menos, se toca el botón de al lado.

Creen también `src/components/ui/Etiqueta.astro`. **Mañana la usan en las tarjetas:**

```
---
// Etiqueta.astro
// Pildora de metadato: categoria, tiempo, dificultad.
const { tono = "neutro" } = Astro.props;
---

<span class={`etiqueta etiqueta--${tono}`}><slot /></span>

<style>
  .etiqueta {
    display: inline-flex;
    align-items: center;
    gap: var(--esp-1);
    font-size: var(--texto-s);
    line-height: 1;
    padding: 0.4rem 0.7rem;
    border-radius: var(--radio-pildora);
    border: 1px solid var(--color-borde);
    background: var(--color-fondo);
    color: var(--color-texto-suave);
    white-space: nowrap;
  }

  .etiqueta--marca {
    background: var(--color-marca-suave);
    border-color: var(--color-marca-borde);
    color: var(--color-marca-oscuro);
    font-weight: 600;
  }
</style>
```

> Si su `variables.css` no tiene alguna variable (`--sombra-1`, `--radio-pildora`, `--transicion`...), agréguenla o reemplácenla por un valor. Una variable que no existe **no da error**: la propiedad simplemente no se aplica.

- [ ] **Listo cuando:** los dos archivos existen en `src/components/ui/` y `npm run dev` no muestra errores.

Paso 3 · La portada (`index.astro`)

Reemplacen `src/pages/index.astro`. Pongan su imagen en `public/img/` (una ilustración, una foto propia o un SVG; si aún no tienen, cualquier imagen de prueba con su `alt`).

```
---
import Layout from "../layouts/Layout.astro";
import Boton from "../components/ui/Boton.astro";

const base = import.meta.env.BASE_URL;
---

<Layout
  titulo="SaborU — Recetas rapidas para quien cocina solo"
  descripcion="Catalogo de recetas rapidas, con lista clara de ingredientes y tiempos reales, para
estudiantes que cocinan solos y con poco tiempo."
  activo="inicio"
  ancho="ancho"
>
  <!-- ============ Portada ============ -->
  <section class="portada">
    <div class="contenedor portada__interior">
      <div class="portada__texto">
        <p class="rotulo">Cocina de estudiante</p>
        <h1>Decide que cocinar esta noche en menos de un minuto</h1>
        <p class="portada__entrada">
          Recetas con lista clara de ingredientes, tiempo real de preparacion y
          pasos que se leen con las manos ocupadas. Sin videos de ocho minutos.
        </p>
        <div class="portada__acciones">
          <Boton href={`${base}catalogo`} tamano="grande">Ver el catalogo</Boton>
          <Boton href={`${base}catalogo`} variante="secundario" tamano="grande">
            Receta aleatoria
          </Boton>
        </div>
        <ul class="portada__datos">
          <li><strong>6</strong> recetas de prueba</li>
          <li><strong>15 min</strong> la mas rapida</li>
          <li><strong>5</strong> ingredientes de media</li>
        </ul>
      </div>

      <div class="portada__medio">
        <img
          src={`${base}img/portada.svg`}
          alt="Ilustracion de un plato servido sobre una mesa"
          width="640"
          height="480"
          fetchpriority="high"
        />
      </div>
    </div>
  </section>

  <!-- ============ Destacadas ============ -->
  <!-- Las tarjetas llegan en la clase 5 con Card y CardGrid. -->
  <section class="contenedor seccion">
    <div class="seccion__encabezado">
      <div>
        <p class="rotulo">Para empezar</p>
        <h2>Tres recetas para una noche de semana</h2>
      </div>
```

```
      <Boton href={`${base}catalogo`} variante="fantasma">Ver todas</Boton>
    </div>
    <p class="pendiente">Aqui van las tres recetas destacadas (clase 5).</p>
  </section>

  <!-- ============ Como funciona ============ -->
  <section class="contenedor seccion">
    <p class="rotulo">Como funciona</p>
    <h2>Tres pasos y la cocina encendida</h2>

    <ol class="pasos">
      <li class="paso">
        <span class="paso__numero">1</span>
        <h3>Mira lo que tienes</h3>
        <p>Filtra por ingrediente principal y por el tiempo del que dispones.</p>
      </li>
      <li class="paso">
        <span class="paso__numero">2</span>
        <h3>Abre la receta</h3>
        <p>Ingredientes arriba, pasos numerados abajo y letra que se lee de lejos.</p>
      </li>
      <li class="paso">
        <span class="paso__numero">3</span>
        <h3>Guarda la que te sirvio</h3>
        <p>Lo que te gusto queda en favoritos para la proxima vez.</p>
      </li>
    </ol>
  </section>
</Layout>

<style>
  /* ---- Portada ---- */
  .portada {
    background: linear-gradient(180deg, var(--color-marca-suave) 0%, var(--color-lienzo) 100%);
    border-bottom: 1px solid var(--color-borde);
  }

  .portada__interior {
    display: grid;
    gap: var(--esp-5);
    padding-top: var(--esp-6);
    padding-bottom: var(--esp-6);
    align-items: center;
  }

  @media (min-width: 900px) {
    .portada__interior {
      grid-template-columns: 1.05fr 0.95fr;
      gap: var(--esp-6);
      padding-top: var(--esp-7);
      padding-bottom: var(--esp-7);
    }
  }

  .portada__texto h1 { max-width: 20ch; }

  .portada__entrada {
```

```
  font-size: var(--texto-m);
  color: var(--color-texto-suave);
  max-width: 46ch;
}

.portada__acciones {
  display: flex;
  flex-wrap: wrap;
  gap: var(--esp-2);
  margin-bottom: var(--esp-5);
}

.portada__datos {
  display: flex;
  flex-wrap: wrap;
  gap: var(--esp-4);
  list-style: none;
  margin: 0;
  padding: var(--esp-3) 0 0;
  border-top: 1px solid var(--color-marca-borde);
  font-size: var(--texto-s);
  color: var(--color-texto-suave);
}

.portada__datos strong {
  display: block;
  font-family: var(--fuente);
  font-size: var(--texto-l);
  color: var(--color-texto);
  line-height: 1.2;
}

.portada__medio img {
  width: 100%;
  height: auto;
  border-radius: var(--radio-l);
  box-shadow: var(--sombra-3);
}

/* ---- Secciones ---- */
.seccion { padding-top: var(--esp-6); }

.seccion__encabezado {
  display: flex;
  flex-wrap: wrap;
  align-items: flex-end;
  justify-content: space-between;
  gap: var(--esp-3);
  margin-bottom: var(--esp-4);
}

.seccion__encabezado h2 { margin: 0; }

.pendiente {
  padding: var(--esp-5);
  border: 2px dashed var(--color-borde-fuerte);
  border-radius: var(--radio-m);
```

```
    color: var(--color-texto-tenue);
    text-align: center;
    margin: 0;
  }

  /* ---- Pasos ---- */
  .pasos {
    display: grid;
    gap: var(--esp-4);
    list-style: none;
    margin: var(--esp-4) 0 0;
    padding: 0;
  }

  @media (min-width: 768px) {
    .pasos { grid-template-columns: repeat(3, 1fr); }
  }

  .paso {
    background: var(--color-fondo);
    border: 1px solid var(--color-borde);
    border-radius: var(--radio-m);
    padding: var(--esp-4);
  }

  .paso__numero {
    display: grid;
    place-items: center;
    width: 36px;
    height: 36px;
    border-radius: var(--radio-pildora);
    background: var(--color-marca-suave);
    color: var(--color-marca-oscuro);
    font-family: var(--fuente);
    font-weight: 600;
    margin-bottom: var(--esp-3);
  }

  .paso h3 { margin-bottom: var(--esp-1); }

  .paso p {
    margin: 0;
    color: var(--color-texto-suave);
    font-size: var(--texto-s);
  }
</style>
```

Lo que deben cambiar: titular, textos, acciones, datos e imagen, **según su mockup**. Si su mockup no tiene "Cómo funciona", reemplacen esa sección por la que sí tenga.

Comparen con su mockup, bloque por bloque:

En el mockup	En el código
Texto a la izquierda, imagen a la derecha	`.portada__interior`: Grid de 2 columnas desde 900 px
Dos botones uno al lado del otro	`.portada__acciones`: Flexbox con `flex-wrap`

En el mockup	En el código
Tres tarjetas en fila	`.pasos`: Grid de 3 columnas desde 768 px
Contenido destacado	Recuadro punteado: se llena mañana con `Card`

Pregunta para responder en el README (2 líneas): achiquen la ventana poco a poco. ¿En qué ancho pasa la portada de una a dos columnas, y qué línea del CSS lo decide?

Prueba obligatoria: cambien un enlace a `href="/catalogo"` (sin `base`), hagan clic y vean el 404. Vuelvan a poner `href={`${base}catalogo`}`. Ningún enlace del proyecto se escribe con `/` a mano.

- [ ] **Listo cuando:** la portada se parece a su mockup en escritorio y en el celular.

Paso 4 · Sus seis rutas

En Astro cada archivo de `src/pages/` es una dirección del sitio. Creen **todas** las páginas de su mapa de pantallas, aunque queden vacías. Cada una con su `activo`, que debe coincidir con el que usa el Header.

Plantilla para las que se construyen después:

```
---
import Layout from "../layouts/Layout.astro";
---

<Layout
  titulo="Catalogo — SaborU"
  descripcion="Catalogo de recetas rapidas con tiempo y dificultad visibles."
  activo="catalogo"
>
  <section class="contenedor">
    <p class="rotulo">Catalogo</p>
    <h1>Que puedes cocinar hoy</h1>
    <p class="entrada">Filtra por categoria o por dificultad.</p>

    <div class="pendiente">Se construye en la clase 5.</div>
  </section>
</Layout>

<style>
  .entrada {
    max-width: var(--ancho-lectura);
    color: var(--color-texto-suave);
  }

  .pendiente {
    margin-top: var(--esp-5);
    padding: var(--esp-6) var(--esp-4);
    border: 2px dashed var(--color-borde-fuerte);
    border-radius: var(--radio-m);
    color: var(--color-texto-tenue);
    text-align: center;
  }
</style>
```

La página **Acerca de** (o la de su proyecto que sea solo texto) sí queda terminada: problema, para quién es, decisiones de diseño y créditos de la API. Usen un Grid de dos columnas desde 900 px con una ficha lateral `position: sticky`. El código completo de referencia está en el PDF del Taller 4.

- [ ] **Listo cuando:** ningún enlace del Header ni del Footer da 404, y la página marcada cambia al navegar.

Paso 5 · Prueba a 320 px (RNF-1)

1. `F12` → icono de celular → ancho **320 px**.
2. Recorran **todas** las páginas.
3. En la consola de cada página:

```
document.documentElement.scrollWidth === document.documentElement.clientWidth
```

Debe responder `true`. Si sale `false`, algo es más ancho que la pantalla: inspeccionen cuál se sale.

- [ ] **Listo cuando:** `true` en todas las páginas.

Paso 6 · Construir y publicar

```
npm run build
```

Debe terminar sin errores, con una página por cada archivo de `src/pages/`.

```
git add .
git commit -m "feat: layout responsive, navegacion y paginas base"
git push
```

Abran **Actions**, esperen el check verde y recarguen la URL publicada. Ábranla también en un celular real.

- [ ] **Listo cuando:** la URL publicada se ve como su mockup.

Los dos integrantes hacen commits. Repártanse los pasos: por ejemplo, uno hace Footer y rutas, y el otro Boton, Etiqueta y portada.

Entrega (antes de la clase del 13-nov)

En el aula virtual o en el README:

1. URL publicada.
2. Dos capturas de la portada: a 320 px y en escritorio.
3. La respuesta de la pregunta del Paso 3.
4. Enlace al último commit. En **Insights** → **Contributors** deben aparecer los dos integrantes.

Al empezar la clase de mañana el profesor abre la URL de cada grupo y prueba un enlace del Header y uno del Footer.

Calificación

Cuenta dentro de la rúbrica de 100 puntos del Taller 4 (PDF). Además:

- Un `npm run build` que falla en Actions resta 15 puntos del criterio "Sitio publicado".
- Reto opcional (+5): que el Header calcule la página activa solo, con `Astro.url.pathname`, en vez de recibir la prop `activo`.

Errores frecuentes

Síntoma	Solución
El código del frontmatter aparece como texto en la página	Faltan las líneas `---` arriba y abajo del bloque de `import`
`Unexpected token` y luego muchos errores de etiquetas sin cerrar	Arreglen **el primer** error y guarden. Suele ser una etiqueta sin nombre (`< id=...>`) o `class=list` en lugar de `class:list`
`Could not resolve "../components/..."`	Ruta del `import` (`components`, no `component`) o mayúsculas del nombre del archivo
`[lightningcss minify] Unexpected end of input`	Un selector o una llave mal escrita en un CSS. Busquen `:{` o una `}` que falta
Un color o un tamaño no se aplica y no hay error	El nombre de la variable no coincide (mayúsculas, guiones) o no existe en `variables.css`
Una clase con modificador no se aplica	Doble guion: `principal--ancho`, `boton--primario`
404 al hacer clic en un enlace	La ruta está escrita con `/` a mano. Usen `` `${base}ruta` ``
La página activa no cambia	El `activo` de la página no coincide con el del arreglo del Header
El sitio publicado se ve sin estilos	Falta `base` en `astro.config.mjs`
Desplazamiento horizontal a 320 px	Un elemento más ancho que la pantalla, casi siempre una imagen sin `width: 100%`
Actions en rojo	Corran `npm run build` antes del push y lean el registro de Actions